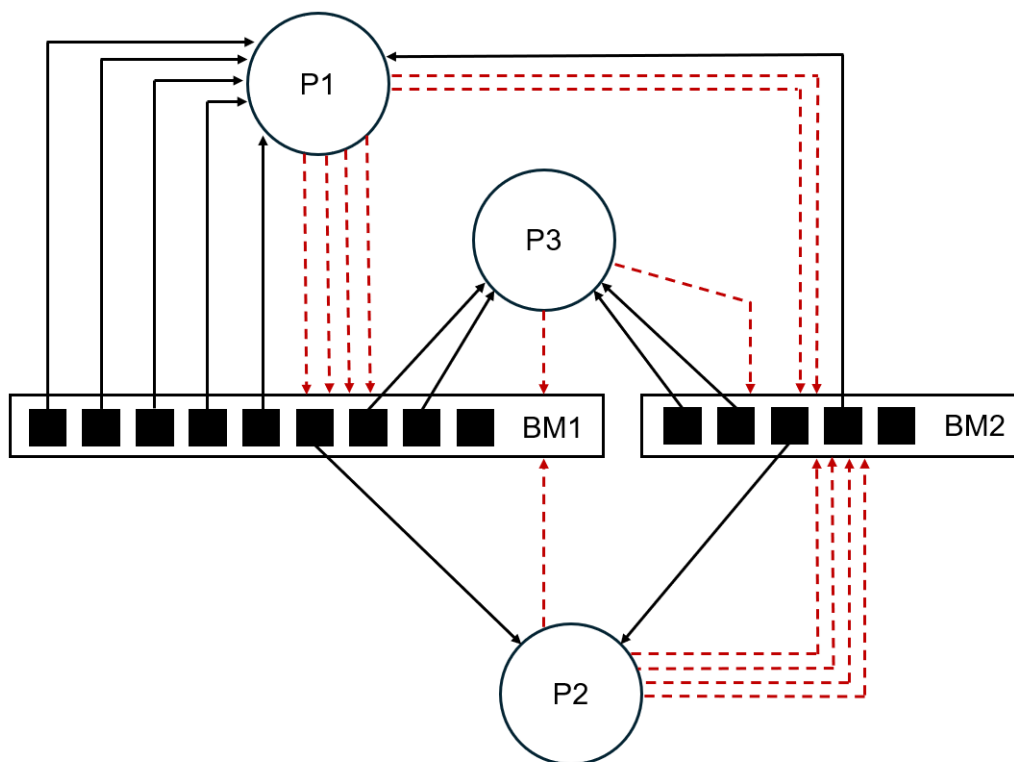


Aufgabe 4.1: Betriebsmittelverwaltung (Tafelübung)

- Nennen Sie Beispiele für Betriebsmittel. Nach welchen Kriterien lassen sie sich klassifizieren?
- Untersuchen Sie den folgenden Betriebsmittelgraphen:



Bestimmen Sie die Matrizen der Belegung B , der Gesamtanforderungen G , sowie die Vektoren der freien Betriebsmittel f und grundsätzlich verfügbaren Betriebsmittel v .

- Wie sieht die Restanforderungs-Matrix R aus?
- Sind im System genügend Ressourcen vorhanden, um theoretisch alle Anforderungen zu erfüllen?
- Erläutern Sie, was ein sicherer Zustand ist und prüfen Sie, ob es sich hier um einen solchen handelt!
- Wie sieht es aus, wenn $f = (2 \ 1)$ ist?
- Was kann man üblicherweise tun, wenn sich ein System in einem Deadlock befindet?

Aufgabe 4.2: Verklemmung

(Tafelübung)

An einem Frühstückstisch in einer WG (Uli, Shanti, Gwen und Lara) gibt es Käse, Toast, Marmelade, Brot und Peanutbutter. Jeden Morgen entsteht ein erbitterter Kampf um die Ressourcen. Die Auseinandersetzung am Frühstückstisch kann näherungsweise durch folgendes C-Programm beschrieben werden:

Es sei ein System gegeben, bei welchem jeder Prozess 2 Befehle ausführen kann, bevor zum nächsten Prozess gewechselt wird. Die Reihenfolge der Ausführung soll wie unten Uli→Shanti→Lara→Gwen→Uli... sein.

- Simulieren Sie den Frühstückstisch. Geben Sie in jedem Schritt an, wer welche Ressource hat oder freigibt.
- Wie sieht der Ablauf aus, wenn jeder Prozess nur einen Befehl ausführen kann, bevor zum nächsten Prozess gewechselt wird?
- In der Vorlesung wurden im Kontext der Betriebsmittelverwaltung zwei verschiedene Typen von Graphen vorgestellt, der Wartegraph und der Betriebsmittelgraph. Wie unterscheiden sich diese?
- Zeichnen Sie den Betriebsmittelgraph wie er am Ende des Ablaufs aus Teil b. aussieht.
- Welche Bedingungen sind notwendig und welche hinreichend, damit eine Verklemmung auftreten kann?

```
// Ressourcen:
// Von jeder Ressource gibt es genau eine Instanz,
// die nicht verbraucht wird
single_unit_R peanutbutter, jelly, cheese, toast, bread;

// Uli - Prozess 1:
int main() {
    allocate_r(&cheese);
    allocate_r(&jelly);
    allocate_r(&bread);
    eat(); // Brot mit Jelly und Cheese
    release_r(&bread);
    release_r(&jelly);
    release_r(&cheese);
    return 0;
}

// Shanti - Prozess 2:
int main() {
    allocate_r(&peanutbutter);
    allocate_r(&bread);
    eat(); // Peanutbutter & Brot
    release_r(&bread);
    release_r(&peanutbutter);
    return 0;
}
```

```
// Lara - Prozess 3:
int main() {
    allocate_r(&toast);
    allocate_r(&cheese);
    eat(); // Käse-Toast
    release_r(&cheese);
    release_r(&toast);
    return 0;
}

// Gwendolin - Prozess 4:
int main() {
    allocate_r(&jelly);
    allocate_r(&peanutbutter);
    allocate_r(&toast);
    eat(); // it's Peanut Butter Jelly Time!!
    release_r(&toast);
    release_r(&peanutbutter);
    release_r(&jelly);
    return 0;
}
```

Aufgabe 4.3: Handsimulation des Bankieralgorithmus (Tafelübung)

Der Bankieralgorithmus kontrolliert Ressourcenallokationen, damit keine unsicheren Zustände auftreten.¹
Gegeben ist die folgende verzahnte Ausführung der vier Prozesse P_1 , P_2 , P_3 und P_4 :

	Zeit	Aktion		Zeit	Aktion
P_1	1	allocate_r(A, 3);	P_1	15	allocate_r(D, 1);
	2	allocate_r(D, 2);		16	release_r(C, 3);
P_2	3	allocate_r(B, 3);	P_4	17	allocate_r(D, 2);
	4	allocate_r(D, 1);		18	release_r(B, 1);
P_3	5	allocate_r(C, 3);	P_3	19	release_r(B, 2);
	6	allocate_r(B, 2);		20	exit();
P_1	7	release_r(A, 3);	P_1	21	release_r(D, 3);
	8	allocate_r(C, 3);		22	exit();
P_4	9	allocate_r(B, 1);	P_2	23	release_r(A, 1);
	10	allocate_r(D, 1);		24	release_r(D, 1);
P_3	11	allocate_r(C, 1);	P_4	25	exit();
	12	release_r(C, 4);		26	release_r(D, 3);
P_2	13	allocate_r(A, 1);		27	exit();
	14	release_r(B, 3);			

Die Funktionen `allocate_r` und `release_r` erhalten hierbei jeweils die Kennung für eines der Betriebsmittel A, B, C oder D und die angeforderte bzw. freizugebende Anzahl als Parameter. Von jedem Betriebsmittel sind zu Beginn 4 Einheiten vorhanden und nicht belegt.

Führen Sie eine Handsimulation für den gegebenen Ablauf durch. Geben Sie dabei zu jedem Zeitpunkt die aktuellen Belegungen, die Restanforderungen und die freien Betriebsmittel in der aus der Vorlesung bekannten Matrixschreibweise an, prüfen Sie mit dem Bankieralgorithmus, ob die Allokation zu einem unsicheren Zustand führt oder nicht. Tritt ein unsicherer Zustand auf oder sind zum Zeitpunkt der Anfrage nicht genug Betriebsmittel vorhanden, wird der dazugehörige Prozess bis zum Ende der Handsimulation blockiert. Blockierte Prozesse geben ihre Betriebsmittel *nicht* frei.

¹Dieser Ansatz ist bei Banken inzwischen scheinbar in Vergessenheit geraten, kommt dort heute doch bevorzugt eine andere Strategie zum Einsatz: [http://de.wikipedia.org/wiki/Bail-out_\(Wirtschaft\)](http://de.wikipedia.org/wiki/Bail-out_(Wirtschaft))